

ClassRoomPB

Sistema Acadêmico — ES2

Relatório de Processo da Release 1

Sprint 1 + Sprint 2

Tecnologia: Java | Arquitetura: MVC | Persistência: JSON

Equipe: Eric Natan · José Eudson · Rui Fernando

2026

1. Visão Geral da Release 1

A Release 1 do sistema ClassRoomPB compreende duas sprints de desenvolvimento, realizadas entre 12/05/2026 e 01/06/2026. O objetivo desta release é entregar os módulos fundamentais do sistema: gestão de usuários, autenticação, controle de acesso, estrutura curricular (cursos, disciplinas e períodos letivos) e gerenciamento de turmas, incluindo validações de integridade e regras de negócio associadas.

Sprint 1	Sprint 2	RFs Totais	Período
RF01–RF09 (9 RFs)	RF10–RF14 (5 RFs)	14 Requisitos Funcionais	12/05 – 01/06/2026

2. Tecnologias Utilizadas

Tecnologia / Ferramenta	Detalhe
Linguagem	Java
Arquitetura	MVC (Model, View, Controller)
Build / Deps	Maven
Testes	JUnit 5 (junit-jupiter)
Cobertura	JaCoCo
Persistência	JSON (arquivos .json por entidade)
Mocks	Mockito 4.11.0

2.1 Dependências Principais (pom.xml)

Dependência	Versão	Uso
Jackson Databind	2.15.2	Serialização/deserialização JSON
Jackson JSR310	2.15.2	Suporte a tipos de data Java 8
JUnit Jupiter	5.10.0	Framework de testes
Mockito	4.11.0	Mocks para testes unitários (Java 8+)
Byte Buddy	1.17.8	Suporte de instrumentação para JDKs mais novos (22/23)
JaCoCo	0.8.10	Cobertura de testes
Maven Shade Plugin	3.2.4	Empacotamento do JAR com dependências

3. Processo de Desenvolvimento e Organização da Equipe

A equipe adotou uma estratégia de distribuição de tarefas baseada em estimativa de horas por task, com o objetivo central de equilibrar a carga de trabalho entre todos os membros e evitar sobrecarga individual. Antes do início de cada sprint, a equipe se reuniu para discutir cada requisito funcional, decompô-lo em tasks menores e atribuir a cada task uma estimativa de horas de trabalho.

Para manter a rastreabilidade entre planejamento e execução, a equipe utilizou o Azure DevOps como ferramenta de gestão. Cada requisito funcional foi cadastrado como um item de trabalho no Azure, e na descrição de cada RF foi adicionado o link direto para o commit correspondente no repositório do GitHub onde aquela funcionalidade foi implementada.

3.1 Sprint 1 (12/05/2026 – 21/05/2026)

Gerente da Sprint: Eric Natan Batista Torres

Membro	RF / Tasks	Entregas
Eric Natan	RF01, RF02, RF04 / Tasks 1.1.1–1.1.5, 1.1.8–1.1.9	Modelagem de usuários, login, validação de duplicidade, testes
José Eudson	RF03, RF05, RF06 / Tasks 1.1.6–1.1.7, 1.1.10–1.1.15	Perfis de acesso, cadastro de cursos e disciplinas, testes
Rui Fernando	RF07, RF08, RF09 / Tasks 1.1.16–1.1.22	Atributos de disciplina, períodos letivos, ativação/encerramento, testes

Atribuições de Eric Natan (Sprint 1)

Implementação das classes (Model e Repository):

- **Usuario:** Classe base abstrata que centraliza os atributos comuns de todos os usuários do sistema (matrícula, nome, e-mail, senha e tipo).
- **Aluno, Professor, Coordenador, Administrador:** Subclasses concretas que estendem Usuario e definem automaticamente o TipoUsuario correspondente ao instanciar.
- **UsuarioService:** Serviço responsável por orquestrar o fluxo de cadastro, login, edição e exclusão de usuários, delegando a persistência ao UsuarioRepository.
- **UsuarioRepository:** Repositório com persistência JSON, métodos de busca por matrícula e por e-mail (case-insensitive) e verificação de duplicidade (existePorMatricula, existePorEmail).
- **MatriculaGenerator:** Utilitário que gera matrículas sequenciais únicas no formato A{NNNN}, P{NNNN}, C{NNNN} e AD{NNNN}.
- **CadastroDuplicadoException:** Exceção de domínio (RuntimeException) que informa claramente quando há tentativa de cadastro com matrícula ou e-mail já existente.

Testes Unitários:

- `UsuarioRepositoryTest`: Cobertura de persistência em disco, operações de CRUD, busca por matrícula e e-mail, e todos os cenários de RF04 (prevenção de duplicidade).
- `CadastroDuplicadoExceptionTest`: Validação do campo violado (MATRICULA ou EMAIL), valor duplicado armazenado e mensagens exibidas ao usuário.

Implementação de Login (RF02):

- Método `login()` no `UsuarioService` que autentica o usuário por matrícula ou e-mail combinado com senha.
- Interface de login no Main que captura as credenciais e redireciona o usuário autenticado para o controlador do seu respectivo perfil.

Atribuições de José Eudson (Sprint 1)

Implementação da Validação de Perfis de Acesso (RF03):

- `PerfilAcessoService`: Serviço que valida se o usuário logado possui o `TipoUsuario` necessário, lançando exceção com mensagem descritiva em caso de acesso não autorizado.
- Integração nos controladores de cada perfil (`AlunoController`, `ProfessorController`, `CoordenadorController`, `AdminController`).
- Testes unitários para cenários de acesso autorizado e acesso negado para cada tipo de perfil.

Cadastro de Cursos (RF05):

- `Curso`: Classe de modelo com código, nome e carga horária.
- `CursoRepository`: Persistência em `cursos.json` com verificação de unicidade por código.
- `CursoService`: Orquestra cadastro, edição, exclusão e listagem de cursos.
- `AdminController`: Menu interativo para gerenciar usuários e cursos.
- Testes unitários: cadastro de cursos, detecção de código duplicado e listagem.

Cadastro de Disciplinas (RF06):

- `Disciplina`: Classe de modelo com código, nome, carga horária, créditos e lista de pré-requisitos opcionais.
- `DisciplinaRepository`: Persistência em `disciplinas.json` com verificação de unicidade por código.
- `DisciplinaService`: Fluxo de cadastro e listagem de disciplinas.
- `CoordenadorController`: Menu para cadastrar disciplinas e gerenciar períodos letivos.
- Testes unitários para cadastro válido, rejeição de código duplicado e consistência dos atributos persistidos.

Atribuições de Rui Fernando (Sprint 1)

Implementação dos Atributos de Disciplina (RF07):

- Atributos completos da entidade `Disciplina`: código, nome, carga horária (inteiro em horas), créditos (inteiro) e lista de pré-requisitos opcionais (`List<String>`).
- Regras de domínio garantem que código e nome não sejam vazios e que carga horária e créditos sejam positivos.

- Método toString() no formato [DISCIPLINA] código - nome (Xh, Y créditos) pré-requisitos: [...].

Cadastro de Períodos Letivos (RF08):

- PeriodoLetivo: Modelo com código (ex.: 2026.2), ano, semestre, datas de início e fim, e flag de status ativo.
- PeriodoLetivoRepository, PeriodoLetivoService e PeriodoLetivoController.
- Testes: cadastro válido, rejeição de código duplicado e validação de campos obrigatórios.

Ativação e Encerramento de Períodos Letivos (RF09):

- Métodos ativarPeriodo() e encerrarPeriodo() no PeriodoLetivoService, alterando o flag ativo e persistindo o estado atualizado.
- Validação de permissão: apenas Coordenadores podem ativar ou encerrar períodos letivos.
- Testes: ativação/encerramento com sucesso, acesso negado e período inexistente.

3.2 Sprint 2 (22/05/2026 – 01/06/2026)

Gerente da Sprint: José Eudson | QA: Rui Fernando | Dev: Eric Natan

Membro	RF / Tasks	Entregas
Eric Natan	RF10, RF11 / Tasks 1.2.1–1.2.5 + Documentação	Modelagem de Turma, oferta de turmas, atributos, testes, documentação
José Eudson	RF12, RF13 / Tasks 1.2.6–1.2.9	Validação de choque de horário, validação de professor responsável, testes
Rui Fernando	RF14 / Tasks 1.2.10–1.2.12	Alteração e cancelamento de turmas, testes

Atribuições de Eric Natan (Sprint 2)

RF10 — Oferta de Turmas:

- Task 1.2.1: Modelagem de classes para Turma (classe Turma e relacionamentos com Disciplina, PeriodoLetivo e Professor).
- Task 1.2.2: Implementação da oferta de turmas pelo Coordenador.
- Task 1.2.3: Testes unitários para oferta de turmas.

RF11 — Atributos da Turma:

- Task 1.2.4: Implementação dos atributos obrigatórios da entidade Turma: professor responsável, limite de vagas, horário e sala.
- Task 1.2.5: Testes unitários para validação dos atributos da turma.

Documentação: Consolidação do relatório de processo da Release 1.

Atribuições de José Eudson (Sprint 2)

RF12 — Validação de Choque de Horário para Professor (RN06):

- Task 1.2.6: Implementação da validação que impede a oferta de turma com horário conflitante para o mesmo professor no mesmo período letivo.
- Task 1.2.7: Testes unitários: sem conflito (oferta permitida), com conflito (oferta rejeitada) e professor sem turmas anteriores.

RF13 — Validação de Professor Responsável:

- Task 1.2.8: Implementação da regra que impede a oferta de turma sem professor responsável válido.
- Task 1.2.9: Testes unitários: tentativa de oferta sem professor e com professor inválido/inexistente.

Atribuições de Rui Fernando (Sprint 2)

RF14 — Alteração e Cancelamento de Turmas:

- Task 1.2.10: Implementação da alteração de turma (professor, vagas, horário, sala) antes do início das aulas.
- Task 1.2.11: Implementação do cancelamento de turma antes da data de início do período letivo.
- Task 1.2.12: Testes unitários: alteração/cancelamento com sucesso, rejeição após início das aulas e turma inexistente.

4. Burndown das Sprints

4.1 Sprint 1 — Burndown

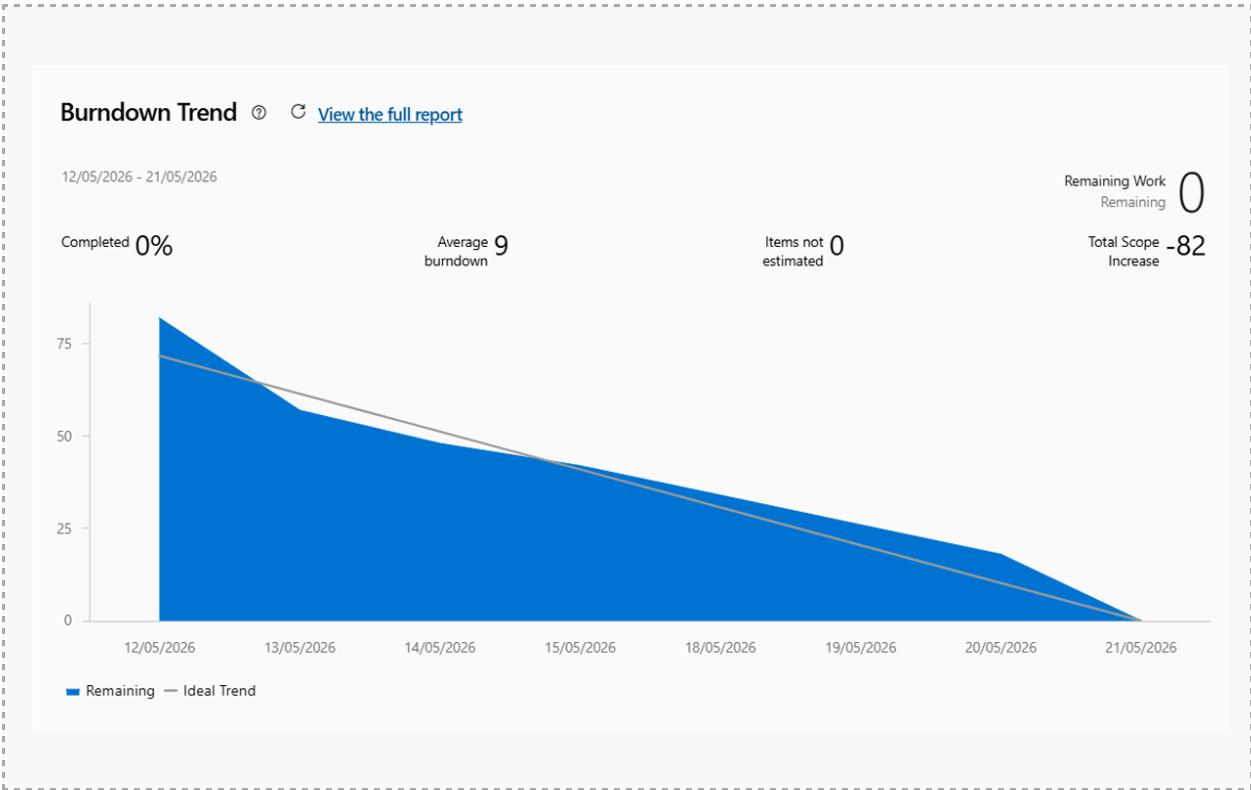


Figura 1: Gráfico de Burndown da Sprint 1

4.2 Sprint 2 — Burndown

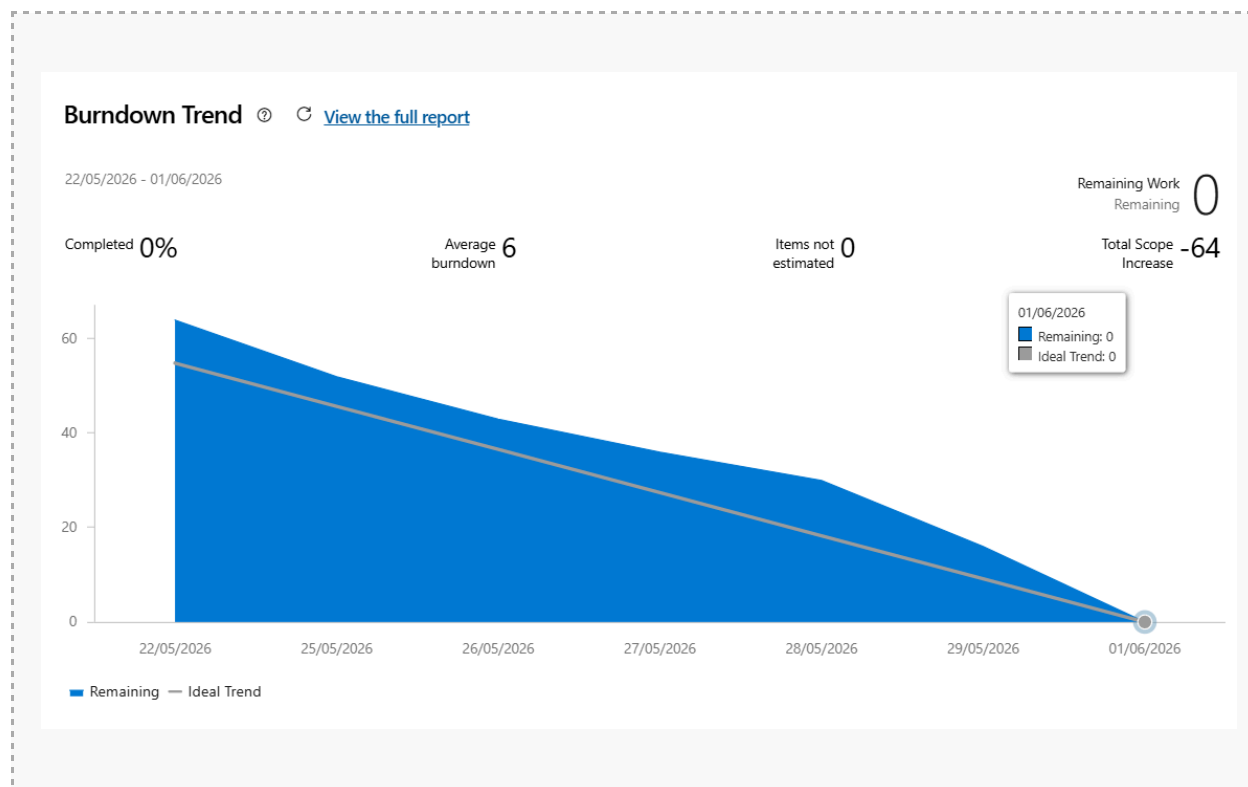


Figura 2: Gráfico de Burndown da Sprint 2

5. Registro das Funcionalidades

5.1 Sprint 1 — RF01 a RF09

RF	Descrição	Responsável
RF01	Cadastro de Alunos, Professores, Coordenadores e Administradores	Eric Natan
RF02	Login com matrícula/e-mail e senha	Eric Natan
RF03	Validação de perfis de acesso com menus diferenciados	José Eudson
RF04	Impedimento de cadastro duplicado por matrícula ou e-mail	Eric Natan
RF05	Cadastro de cursos pelo Administrador	José Eudson
RF06	Cadastro de disciplinas pelo Coordenador	José Eudson
RF07	Atributos completos para disciplinas (código, nome, carga, créditos, pré-requisitos)	Rui Fernando
RF08	Cadastro de períodos letivos pelo Coordenador	Rui Fernando
RF09	Ativação e encerramento de períodos letivos	Rui Fernando

5.2 Sprint 2 — RF10 a RF14

RF	Descrição	Responsável
RF10	Oferta de turmas pelo Coordenador para uma disciplina em um período letivo	Eric Natan
RF11	Atributos obrigatórios da turma: professor responsável, limite de vagas, horário e sala	Eric Natan
RF12	Impedimento de choque de horário para o professor (RN06)	José Eudson
RF13	Impedimento de oferta de turma sem professor responsável	José Eudson
RF14	Alteração ou cancelamento de turma antes do início das aulas	Rui Fernando

6. Detalhamento das Funcionalidades — Sprint 1

6.1 Cadastro de Usuários e Login (RF01 e RF02)

O sistema ClassRoomPB implementa um fluxo de cadastro que suporta quatro perfis distintos de usuário: Aluno, Professor, Coordenador e Administrador. Cada perfil é representado por uma subclasse de `Usuario` que define automaticamente o `TipoUsuario` correspondente, simplificando a lógica de instanciação e garantindo consistência nos dados persistidos.

O processo de cadastro é centralizado no `UsuarioService`, que delega a geração automática de matrícula ao `MatriculaGenerator` — produzindo identificadores no formato `A{NNNN}` para alunos, `P{NNNN}` para professores, `C{NNNN}` para coordenadores e `AD{NNNN}` para administradores — e a persistência ao `UsuarioRepository`, que armazena os dados em arquivo JSON.

Para o login (RF02), o usuário pode se autenticar informando matrícula ou e-mail, combinado com a senha cadastrada. O serviço verifica as credenciais no repositório e, em caso de sucesso, redireciona o usuário autenticado para o controlador correspondente ao seu perfil. Em caso de falha, uma mensagem de erro é exibida sem revelar qual campo está incorreto.

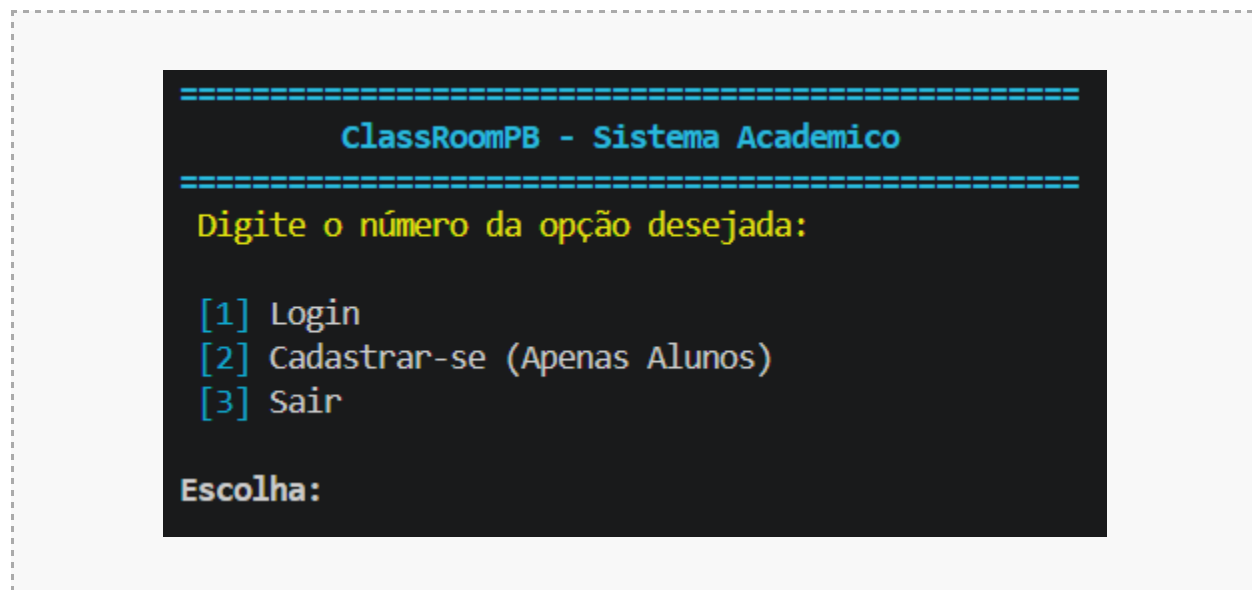


Figura 3: Menu inicial e tela de login do Sistema ClassRoomPB

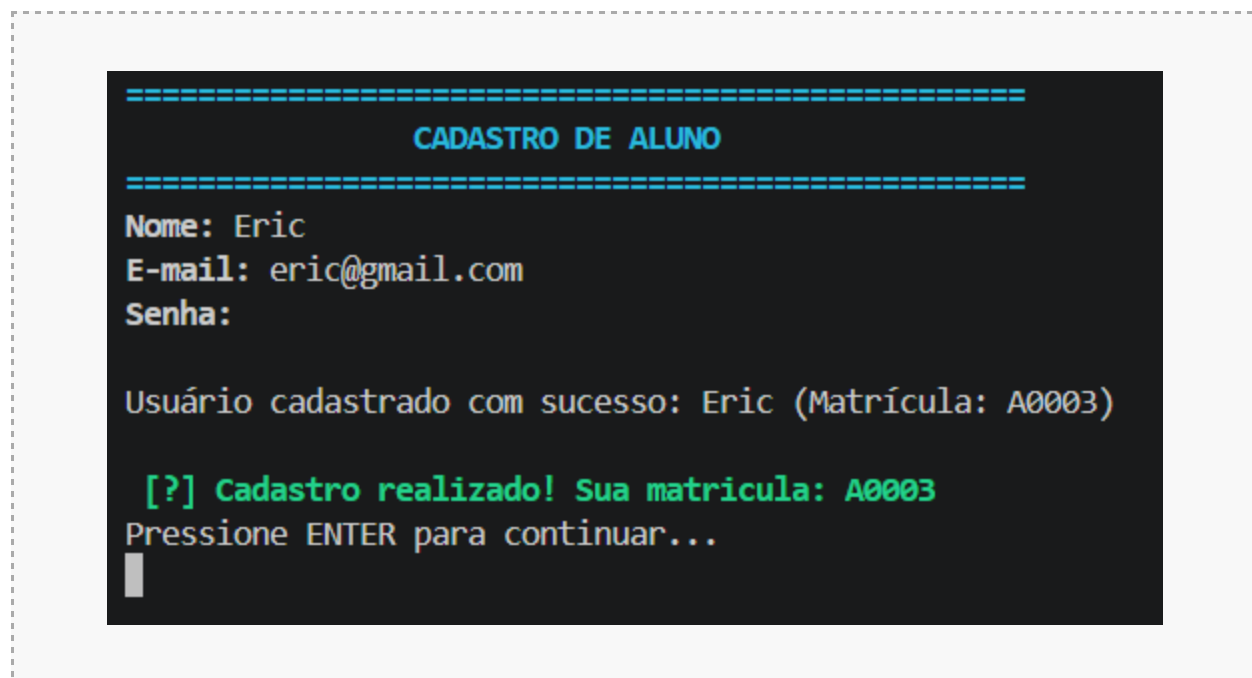


Figura 4: Fluxo de cadastro de Aluno via console

6.2 Validação de Perfis de Acesso (RF03)

O sistema implementa controle de acesso baseado em perfil por meio do `PerfilAcessoService`. Cada controlador (`AlunoController`, `ProfessorController`, `CoordenadorController` e `AdminController`) invoca este serviço no início do seu método `exibirMenu()`, garantindo que somente usuários com o perfil correto possam acessar as funcionalidades disponíveis.

Caso um usuário tente acessar uma área para a qual não possui permissão, o sistema lança uma exceção descritiva e retorna ao fluxo anterior sem expor informações sensíveis.

Os menus apresentados ao usuário variam conforme o perfil: o Aluno acessa matrícula em disciplinas e notas; o Professor registra frequências e notas; o Coordenador gerencia disciplinas, turmas e períodos letivos; o Administrador gerencia usuários e cursos.

```
=====
MENU ADMINISTRADOR
=====
Digite o número da opção desejada:

[1] Gerenciar usuários
[2] Gerenciar cursos
[3] Logout

Escolha: █
```

Figura 5: Menu do Administrador após autenticação

```
=====
MENU COORDENADOR
=====
Digite o número da opção desejada:

[1] Gerenciar disciplinas
[2] Gerenciar período letivo
[3] Gerenciar turmas
[4] Logout

Escolha: █
```

Figura 6: Menu do Coordenador após autenticação

6.3 Impedimento de Cadastro Duplicado (RF04)

Para garantir a integridade dos dados, o sistema impede que dois usuários sejam cadastrados com a mesma matrícula ou o mesmo endereço de e-mail. Essa validação é realizada antes da persistência dos dados e opera de forma case-insensitive para o campo de e-mail (por exemplo, "joao@email.com" e "JOAO@EMAIL.COM" são considerados idênticos).

Quando uma duplicidade é detectada, o sistema lança uma `CadastroDuplicadoException` com a informação exata do campo violado (matrícula ou e-mail) e o valor duplicado:

- Para matrícula duplicada: "Erro: Já existe um usuário com esta matrícula."

- Para e-mail duplicado: "Erro: Já existe um usuário com este e-mail."

A validação abrange todos os tipos de usuário de forma uniforme, e a consistência é mantida mesmo após operações de deleção.

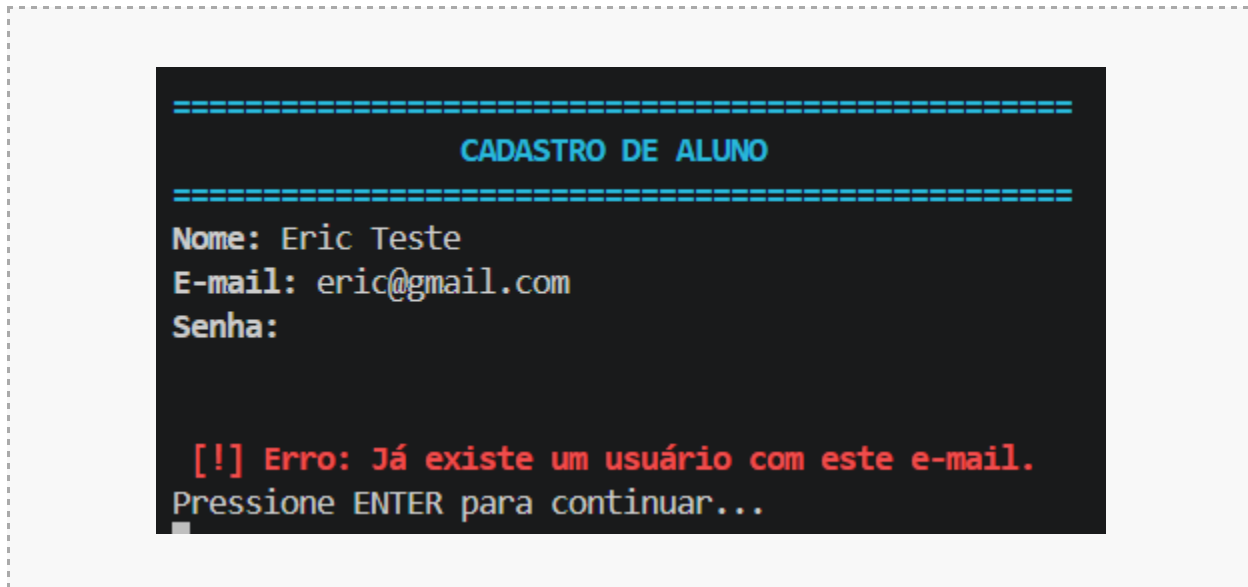


Figura 7: Mensagem de erro ao tentar cadastrar e-mail já existente

6.4 Cadastro de Cursos pelo Administrador (RF05)

O sistema permite ao Administrador cadastrar, editar, listar e remover cursos por meio de um menu interativo no console. Cada curso é definido por um código único, um nome descritivo e a carga horária total em horas.

O fluxo de cadastro é gerenciado pelo AdminController, que coleta os dados via ConsoleUI, delega as validações ao CursoService e persiste os dados por meio do CursoRepository em arquivo JSON (cursos.json). A unicidade do código é verificada pelo serviço antes de qualquer operação de criação, impedindo registros duplicados.

A edição de cursos permite alterar nome e carga horária, mantendo o valor atual para campos deixados em branco pelo administrador. A remoção exige confirmação explícita antes de efetivar a exclusão.

```

=====
                        GERENCIAR CURSOS
=====
    Digite o número da opção desejada:

    [1] Cadastrar novo curso
    [2] Editar curso
    [3] Listar cursos
    [4] Deletar curso
    [5] Voltar

    Escolha: █

```

Figura 8: Menu de gerenciamento de cursos

```

=====
                        LISTA DE CURSOS
=====
+-----+-----+-----+
| Código | Nome                               | Carga Horária |
+-----+-----+-----+
| ADS    | Análise e Desenvolvimento de Sistemas | 3200h          |
| CD     | Ciencia de Dados                     | 3200h          |
| EE     | Engenharia Eletrica                  | 3200h          |
+-----+-----+-----+

Total: 3

[?] Fim da lista.
Pressione ENTER para continuar...
█

```

Figura 9: Listagem dos cursos em tabela

6.5 Cadastro de Disciplinas pelo Coordenador (RF06 e RF07)

O Coordenador tem acesso ao menu de cadastro de disciplinas, onde pode informar código, nome, carga horária (em horas), número de créditos e os pré-requisitos opcionais (informados

como lista separada por vírgulas). Todos esses atributos estão em conformidade com o RF07, que especifica os campos obrigatórios de uma disciplina.

A persistência é realizada pelo `DisciplinaRepository` em arquivo JSON (`disciplinas.json`). O serviço `DisciplinaService` garante que o código da disciplina seja único no sistema, lançando exceção em caso de tentativa de duplicidade.

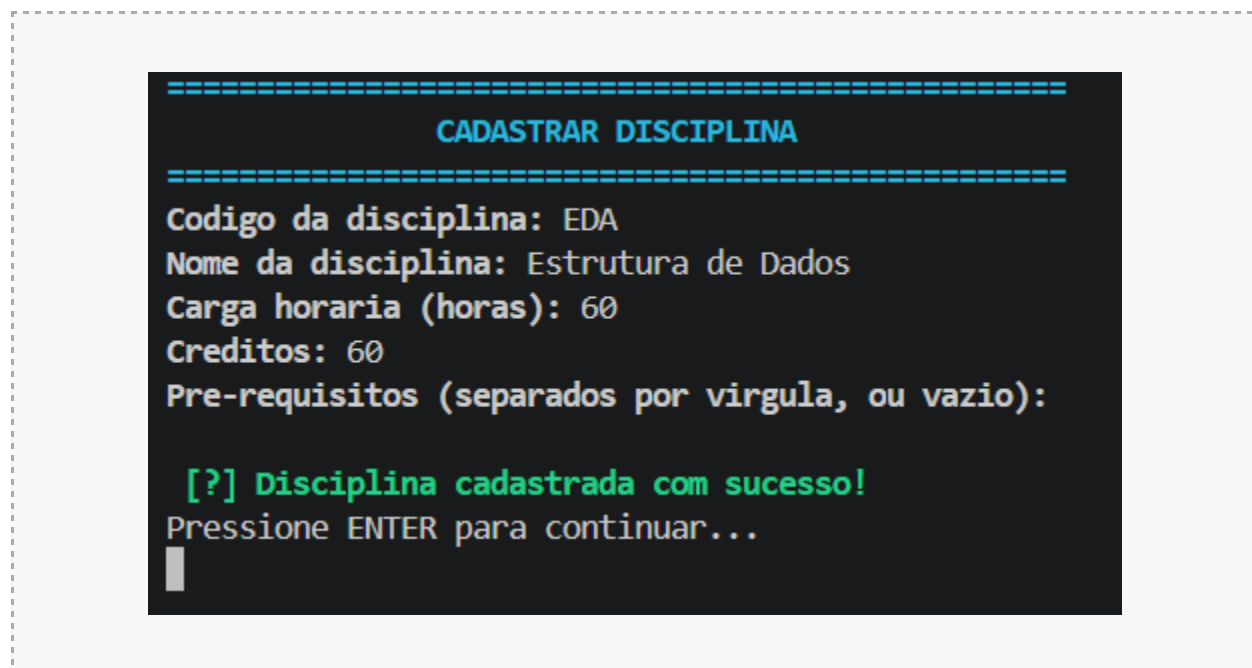


Figura 10: Cadastro de disciplina via menu do Coordenador

6.6 Cadastro e Gestão de Períodos Letivos (RF08 e RF09)

O sistema permite ao Coordenador cadastrar períodos letivos identificados por um código no formato "YYYY.S" (ex.: "2026.2"), informando ano, semestre, datas de início e fim, e o status inicial (ativo ou inativo). O cadastro é gerenciado pelo `PeriodoLetivoService` e os dados são persistidos em arquivo JSON por meio do `PeriodoLetivoRepository`.

Para atender ao RF09, o sistema implementa as operações de ativação e encerramento de períodos. O método `ativarPeriodo()` define o flag ativo como verdadeiro, enquanto o método `encerrarPeriodo()` o define como falso. A validação de permissão garante que apenas Coordenadores possam executar essas operações.

```
=====
                        CADASTRAR PERÍODO LETIVO
=====
Ano: 2028
Semestre (1 ou 2): 1
Data início (dd/mm/aaaa): 10/02/2028
Data fim (dd/mm/aaaa): 18/07/2028
Período ativo? (S/N): s

[?] Período cadastrado com sucesso! Código: 2028.1
Pressione ENTER para continuar...
█
```

Figura 11: Cadastro e ativação de período letivo pelo Coordenador

7. Detalhamento das Funcionalidades — Sprint 2

7.1 Oferta de Turmas pelo Coordenador (RF10 e RF11)

O Coordenador pode ofertar turmas para uma disciplina em um período letivo ativo. Cada turma possui os atributos obrigatórios definidos no RF11: professor responsável, limite de vagas, horário e sala. A oferta é gerenciada pelo TurmaService, que delega validações às regras de negócio aplicáveis antes de persistir a turma via TurmaRepository em turmas.json.

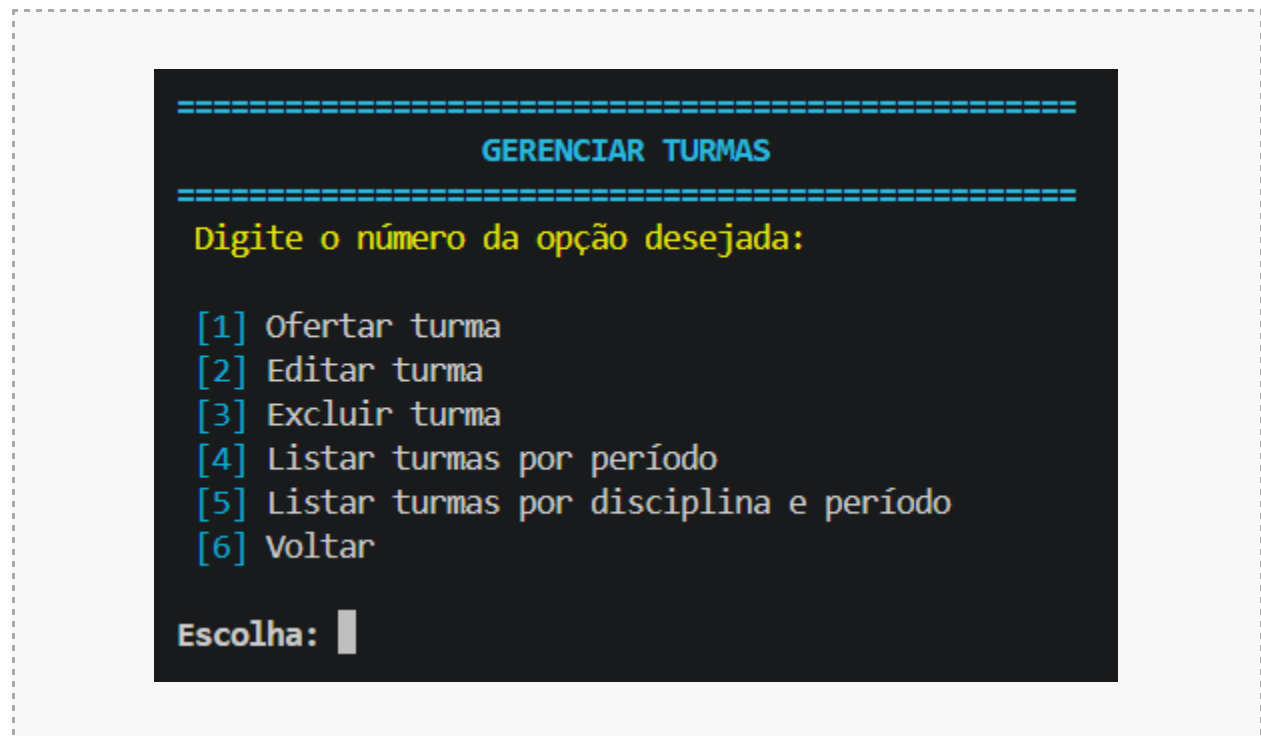


Figura 12: Menu de oferta de turmas pelo Coordenador

```

=====
                        OFERTAR TURMA
=====
Código da disciplina: EDA
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T01
Número de vagas: 45
Horário (ex: Seg/Qua 10h-12h): Seg / Quarta 11h - 13h
Sala (ex: Bloco A - 101): Bloco C - 106
Matrícula do professor responsável: P0002

[?] Turma ofertada com sucesso!
Pressione ENTER para continuar...

```

Figura 13: Formulário de ofertar de turma com atributos obrigatórios

7.2 Impedimento de Choque de Horário para Professor (RF12 — RN06)

Ao realizar a oferta de uma turma, o sistema verifica se o professor designado já possui outra turma com horário conflitante no mesmo período letivo. Caso haja conflito, a oferta é rejeitada com uma mensagem descritiva informando a turma conflitante. Essa validação é implementada no TurmaService e garante que um professor não seja vinculado a duas turmas simultâneas.

```

=====
                        OFERTAR TURMA
=====
Código da disciplina: ES2
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T01
Número de vagas: 35
Horário (ex: Seg/Qua 10h-12h): Seg / Quarta 11h - 13h
Sala (ex: Bloco A - 101): Bloco A - 118
Matrícula do professor responsável: P0002

[!] Erro: RF12/RN06 - O professor 'P0002' já ministra uma turma no horário 'Seg / Quarta 11h - 13h' neste período.
Pressione ENTER para continuar...

```

Figura 14: Mensagem de erro ao tentar ofertar turma com choque de horário para o professor

7.3 Impedimento de Oferta sem Professor Responsável (RF13)

O sistema rejeita a oferta de qualquer turma que não possua um professor responsável designado e válido. A validação é executada antes da persistência, garantindo que nenhuma turma ativa exista no sistema sem responsável docente.

```
=====
                        OFERTAR TURMA
=====
Código da disciplina: ES2
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T01
Número de vagas: 30
Horário (ex: Seg/Qua 10h-12h): Seg / Sex 9h-11h
Sala (ex: Bloco A - 101): Bloco A - 118
Matrícula do professor responsável:

[!] Erro: Não é possível ofertar turma sem professor responsável.
Pressione ENTER para continuar...
█
```

Figura 15: Mensagem de erro ao tentar ofertar turma sem professor responsável

7.4 Alteração e Cancelamento de Turmas (RF14)

O Coordenador pode alterar os atributos de uma turma (professor, vagas, horário e sala) ou cancelar a turma, desde que essas operações sejam realizadas antes da data de início do período letivo ao qual a turma pertence:

- Alteração: aplica apenas as mudanças fornecidas, mantendo os valores anteriores para campos não alterados.
- Cancelamento: a turma é marcada como cancelada e retirada da lista de turmas ativas do período.
- Restrição temporal: qualquer tentativa de alterar ou cancelar uma turma após o início do período letivo é rejeitada com mensagem de erro explicativa.

```
=====
                        EDITAR TURMA
=====
Código da disciplina: EDA
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T01

Dados atuais:
  Vagas   : 45
  Horário: Seg / Quarta 11h - 13h
  Sala    : Bloco C - 106
  Prof    : P0002

Novas vagas (vazio para manter):
Novo horário (vazio para manter):
Nova sala (vazio para manter):
Nova matrícula do professor (vazio para manter):
```

Figura 16: Menu de editar de turma pelo Coordenador

```
=====
      Tem certeza que deseja excluir esta turma?
=====
Digite o número da opção desejada:

[1] Sim, excluir
[2] Não, cancelar

Escolha: 1

[!] Erro: RF14 - Não é possível cancelar uma turma após o início das aulas (início: 2026-01-01).
Pressione ENTER para continuar...
█
```

Figura 17: Mensagem de erro ao tentar excluir turma após início do período letivo

8. Testes Unitários

Todos os requisitos implementados em ambas as sprints possuem cobertura de testes unitários, escritos com o framework JUnit 5 (junit-jupiter) e executados via Maven (mvn test). Os testes foram organizados em classes aninhadas (@Nested) para separar claramente os cenários de cada funcionalidade.

8.1 Testes da Sprint 1

Testes de UsuarioRepository (RF01, RF02 e RF04)

Os testes fazem uso do @TempDir do JUnit 5, que cria uma pasta temporária isolada para cada teste e a remove automaticamente ao final, garantindo que o arquivo usuarios.json real nunca seja afetado durante a execução da suíte. Os cenários cobertos incluem:

- Persistência em disco: salvar e recuperar usuário em nova instância do repositório, persistir múltiplos usuários de tipos diferentes, e iniciar com lista vazia na ausência de arquivo.
- CRUD completo: salvar, buscar por matrícula, buscar por e-mail (case-insensitive), atualizar e deletar usuários — com exceções para operações sobre registros inexistentes.
- RF04 — Prevenção de duplicidade: existePorMatricula e existePorEmail retornam os valores corretos; verificações são case-insensitive para e-mail; após deleção, matrícula e e-mail não são mais detectados como existentes.

Testes de CadastroDuplicadoException (RF04)

- Duplicidade por matrícula: campo armazenado, valor preservado, mensagem correta e compatibilidade com RuntimeException.
- Duplicidade por e-mail: campo armazenado, valor preservado (incluindo casing original), mensagem correta.
- Distinção entre campos: mensagens de MATRICULA e EMAIL são distintas; getCampo() e getValorDuplicado() nunca retornam null.

Testes de RF03, RF05, RF06, RF07, RF08 e RF09

- Acesso autorizado e negado por perfil de usuário (RF03).
- Cadastro de curso com código único, rejeição de duplicidade e listagem (RF05).
- Cadastro de disciplina com todos os atributos, rejeição de código duplicado e consistência da persistência (RF06 e RF07).
- Cadastro de período letivo com datas válidas e código único (RF08).
- Ativação e encerramento de período com persistência de estado; rejeição de operações por usuários sem permissão e para períodos inexistentes (RF09).

8.2 Testes da Sprint 2

Testes de Oferta de Turmas (RF10 e RF11)

- Oferta de turma válida com todos os atributos obrigatórios preenchidos.

- Rejeição de turma com vagas inválidas (zero ou negativas).
- Rejeição de turma com horário ou sala em branco.
- Persistência correta dos dados da turma em turmas.json.

Testes de Choque de Horário para Professor (RF12)

- Oferta permitida quando professor não possui outras turmas no período.
- Oferta rejeitada quando há conflito de horário com turma já existente do mesmo professor.
- Oferta permitida para o mesmo professor em horário diferente no mesmo período.

Testes de Professor Responsável (RF13)

- Oferta rejeitada ao tentar criar turma sem professor designado.
- Oferta rejeitada ao designar professor inexistente no repositório.
- Oferta permitida com professor válido e cadastrado.

Testes de Alteração e Cancelamento de Turmas (RF14)

- Alteração com sucesso de atributos antes do início do período letivo.
- Cancelamento com sucesso antes do início do período letivo.
- Rejeição de alteração após início do período letivo.
- Rejeição de cancelamento após início do período letivo.
- Rejeição de operação em turma inexistente.

8.3 Cobertura de Testes — JaCoCo (Sprint 1)

O relatório de cobertura gerado pelo JaCoCo ao final da Sprint 1 apresenta uma cobertura global de 94% de instruções e 90% de branches, totalizando 2.399 instruções e 214 branches analisados. Os pacotes com cobertura total (100%) são `com.classroompb.model` e `com.classroompb.exception`.

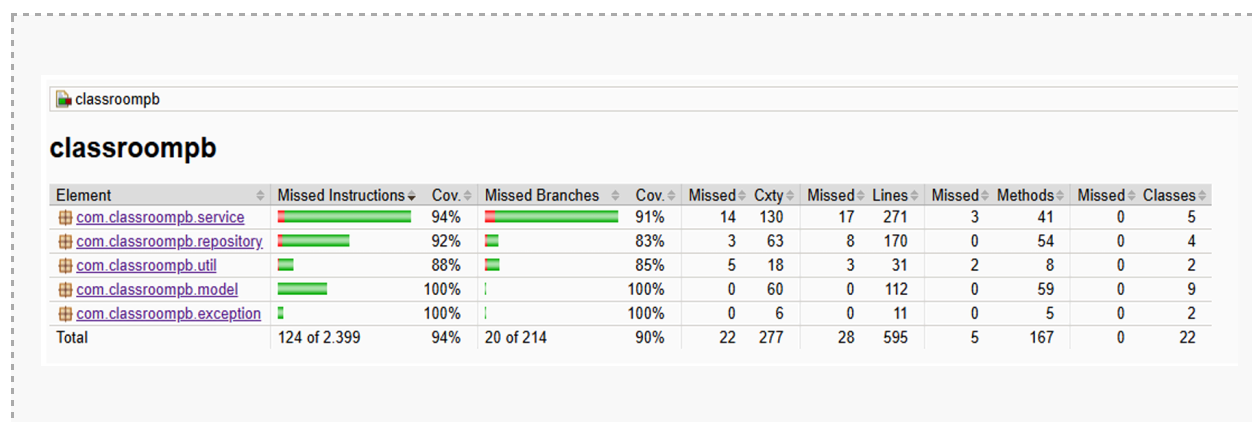


Figura 18: Relatório de cobertura de testes — JaCoCo (Sprint 1)

8.4 Cobertura de Testes — JaCoCo (Sprint 2)

O relatório de cobertura gerado pelo JaCoCo ao final da Sprint 2 apresenta uma cobertura global de 95% de instruções e 85% de branches, totalizando 3611 instruções e 468 branches analisados. Os pacotes com cobertura total (100%) são `com.classroompb.model` e `com.classroompb.exception`.

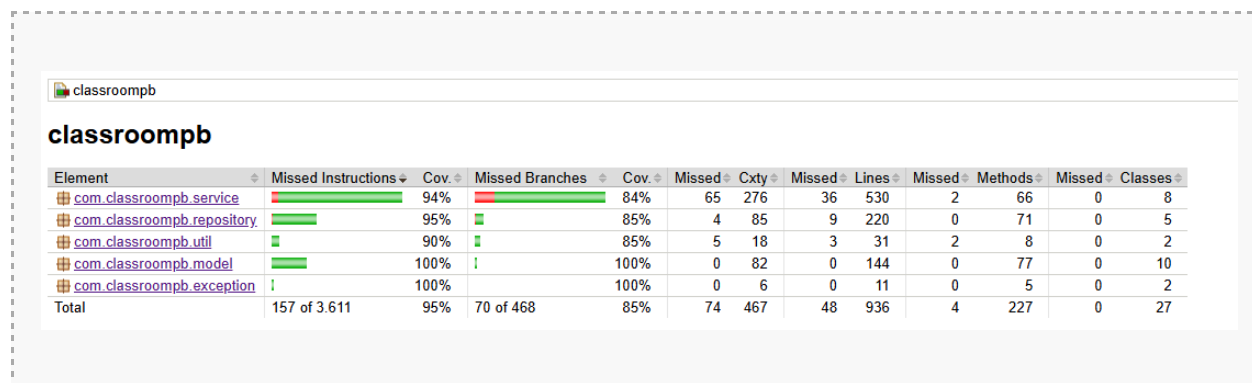


Figura 19: Relatório de cobertura de testes — JaCoCo (Sprint 2)

9. Persistência de Dados

O sistema ClassRoomPB adota JSON como mecanismo de persistência de dados. Cada entidade do domínio é serializada e armazenada em um arquivo .json dedicado, localizado no diretório de dados da aplicação. Essa abordagem elimina a dependência de um sistema de banco de dados externo, simplificando o ambiente de desenvolvimento e execução.

A leitura e escrita dos arquivos é centralizada nas classes Repository de cada entidade, que encapsulam a lógica de serialização e desserialização via JsonUtil. Ao iniciar, cada repositório carrega o arquivo correspondente; ao persistir, sobrescreve o arquivo com o estado atualizado da coleção em memória.

Arquivo	Entidade	Responsável	Sprint
usuarios.json	Usuario (todos os perfis)	Eric Natan	Sprint 1
cursos.json	Curso	José Eudson	Sprint 1
disciplinas.json	Disciplina	José Eudson / Rui Fernando	Sprint 1
periodos_letivos.json	PeriodoLetivo	Rui Fernando	Sprint 1
turmas.json	Turma	Eric Natan	Sprint 2

9.1 Exemplos de Conteúdo dos Arquivos JSON


```
{ } disciplinas.json > ...
1  [ {
2    "codigo" : "ES2",
3    "nome" : "Engenharia de Software 2",
4    "cargaHoraria" : 60,
5    "creditos" : 60,
6    "preRequisitos" : [ "Engenharia de Software 1" ]
7  }, {
8    "codigo" : "GP",
9    "nome" : "Gerencia de Projetos",
10   "cargaHoraria" : 60,
11   "creditos" : 60,
12   "preRequisitos" : [ "Engenharia de Software 1" ]
13 }, {
14   "codigo" : "LP2",
15   "nome" : "Linguagem de Programacao 2",
16   "cargaHoraria" : 60,
17   "creditos" : 60,
18   "preRequisitos" : [ "Linguagem de Programacao 1" ]
19 }, {
20   "codigo" : "EDA",
21   "nome" : "Estrutura de Dados",
22   "cargaHoraria" : 60,
23   "creditos" : 60,
24   "preRequisitos" : [ ]
25 }
```

Figura 20: Conteúdo do arquivo disciplinas.json

```
{ } cursos.json > ...
1  [ {
2    "codigo" : "ADS",
3    "nome" : "Análise e Desenvolvimento de Sistemas",
4    "cargaHoraria" : 3200
5  }, {
6    "codigo" : "CD",
7    "nome" : "Ciencia de Dados",
8    "cargaHoraria" : 3200
9  }, {
10   "codigo" : "EE",
11   "nome" : "Engenharia Eletrica",
12   "cargaHoraria" : 3200
13  } ]
```

Figura 21: Conteúdo do arquivo *cursos.json*

```
{ } periodos_letivos.json > { } 3
1  [ {
2    "codigo" : "2026.1",
3    "ano" : 2026,
4    "semestre" : 1,
5    "dataInicio" : [ 2026, 1, 1 ],
6    "dataFim" : [ 2026, 6, 2 ],
7    "ativo" : true
8  }, {
9    "codigo" : "2026.2",
10   "ano" : 2026,
11   "semestre" : 2,
12   "dataInicio" : [ 2026, 6, 1 ],
13   "dataFim" : [ 2026, 12, 10 ],
14   "ativo" : false
15 }, {
16   "codigo" : "2027.1",
17   "ano" : 2027,
18   "semestre" : 1,
19   "dataInicio" : [ 2027, 1, 1 ],
20   "dataFim" : [ 2027, 6, 1 ],
21   "ativo" : false
22 }, {
23   "codigo" : "2027.2",
24   "ano" : 2027,
25   "semestre" : 2,
26   "dataInicio" : [ 2027, 6, 1 ],
27   "dataFim" : [ 2027, 12, 1 ],
28   "ativo" : false
29 }, {
```

Figura 22: Conteúdo do arquivo periodos_letivos.json

```
{ } usuarios.json > { } 4
1  [ {
2    "matricula" : "AD0001",
3    "nome" : "Administrador",
4    "email" : "admin@gmail.com",
5    "senha" : "admin123",
6    "tipo" : "ADMINISTRADOR"
7  }, {
8    "matricula" : "C0001",
9    "nome" : "coordenador",
10   "email" : "coordenador@gmail.com",
11   "senha" : "coordenador123",
12   "tipo" : "COORDENADOR"
13 }, {
14   "matricula" : "A0001",
15   "nome" : "Aluno teste",
16   "email" : "aluno@gmail.com",
17   "senha" : "alunoteste",
18   "tipo" : "ALUNO"
19 }, {
20   "matricula" : "P0001",
21   "nome" : "Professor",
22   "email" : "professor@gmail.com",
23   "senha" : "professorteste",
24   "tipo" : "PROFESSOR"
25 }, {
26   "matricula" : "P0002",
27   "nome" : "Sabrina",
28   "email" : "sabrina@gmail.com",
29   "senha" : "teste123",
30   "tipo" : "PROFESSOR"
31 }, {
```

Figura 23: Conteúdo do arquivo usuarios.json

```
{ } turmas.json > ...
1  [ {
2    "codigo" : "T01",
3    "codigoDisciplina" : "GP",
4    "codigoPeriodo" : "2026.1",
5    "vagas" : 30,
6    "horario" : "Seg 9h-11h / Qua 11h-13h",
7    "sala" : "Bloco A - 118",
8    "matriculaProfessor" : "P0001"
9  }, {
10   "codigo" : "T01",
11   "codigoDisciplina" : "EDA",
12   "codigoPeriodo" : "2026.1",
13   "vagas" : 45,
14   "horario" : "Seg / Quarta 11h - 13h",
15   "sala" : "Bloco C - 106",
16   "matriculaProfessor" : "P0002"
17 }
```

Figura 24: Conteúdo do arquivo turmas.json